## 实验摘要：

1、 用 Matlab、python 等编程语言实现题目给出的傅里叶变换和傅里叶反变换。

2、 用 Matlab、python 等编程语言观察傅里叶的时移特性，并画出幅度谱和相位谱。

3、 用 Matlab、pthon 等编程语言对系统函数进行部分分式展开求解反变换。

4、 用 Matlab、python 等编程语言画出系统函数，观察零、极点分布，判断系统稳定性。

5、用 Matlab、python 等编程语言求解系统函数的冲击响应和阶跃响应，以及给定输入的零状态响应。

6、提高学生应用编程语言解决信号问题的能力。

## 实验题目：

1、使用Matlab函数计算 $f(t)=e^{-2|t|}$ 的傅里叶变换， $F(\omega)=\frac{1}{1+\omega^2}$ 傅里叶反变换。

参考函数：fourier( )，ifourier( )

2、计算 $f_1(t)=\frac{1}{2}e^{-2t}\varepsilon(t)$ 和 $f_2(t)=\frac{1}{2}e^{-2(t-1)}\varepsilon(t-1)$ 的傅里叶变换，画出其幅度谱和相位谱，并观察傅里叶变换的时移特性。

参考函数：syms( )

3、用部分分式展开法求 $H(s)=\frac{(s+1)(s+4)}{s(s+2)(s+3)}$ 的反变换。

参考函数：因子形式转换多项式conv( )，residue( )

4、画出系统函数 $H_1(s)=\frac{s+2}{s^3+s^2+2s+6}$ 和 $H_2(s)=\frac{s^2+1}{3s^3+5s^2+4s+6}$ 零、极点分布，并判断系统的稳定性。

参考函数：laplace( )，roots( )，pzmap( )，sys( )

5、用MATLAB计算拉普拉斯变换求解 $H(s)=\frac{s}{s^2+3s+2}$ 的冲激响应、阶跃响应，以及激励 $f(t)=\cos(20t)\varepsilon(t)$ 产生的零状态响应，给出运行结果并分析。

参考函数：laplace( )，ilaplace( )，sys( )等

实验内容：

一、 实验基本原理及步骤

1、 计算傅里叶变换和傅里叶反变换

1-1 计算傅里叶变换

1） 首先定义时域和频域符号

2） 表达出时域信号

3） 调用 forier 函数来进行傅里叶变换

4） 打印出原时域信号和变换后的频域信号

具体代码如下：

```
clear;clc
syms t omega
f_origin=exp(-2*abs(t));
f_change = fourier(f_origin, t, omega);
disp('时域信号：');
disp(f_origin);
disp('傅里叶变换后频域信号：')
disp(f_change)
```

1-2 计算傅里叶反变换

1） 首先定义时域和频域符号

2） 表达出频域信号

3） 调用 iforier 函数来进行傅里叶反变换

4） 打印出原频域信号和变换后的时域信号

具体代码如下：

```
clear;clc
syms t omega
f_origin=1/(1+omega^2);
f_change = ifourier(f_origin, omega, t);
disp('频域信号：');
disp(f_origin);
disp('傅里叶反变换后时域信号：')
disp(f_change)
```

2、 计算函数的傅里叶变换并绘制幅度谱和相位谱，观察傅里叶的时移特性

1） 按照问题 1 的方法定义原始时域信号，调用函数进行傅里叶变换。

2） 将得到的符号表达式转变成数值函数，并定义频率范围。

3） 计算两个信号的幅度和相位，根据计算结果画出相应的图形。

具体代码如下：

```
clear; clc;
syms t omega real

% 定义时域信号
f_origin_1 = 0.5 * exp(-2 * t) * heaviside(t);
f_origin_2 = 0.5 * exp(-2 * (t - 1)) * heaviside(t - 1);
```

```
% 计算傅里叶变换
f_change_1 = fourier(f_origin_1, t, omega);
f_change_2 = fourier(f_origin_2, t, omega);

% 转换符号表达式为数值函数
f_change_1_num = matlabFunction(f_change_1, 'Vars', omega);
f_change_2_num = matlabFunction(f_change_2, 'Vars', omega);

% 定义频率范围
omega_values = linspace(-10, 10, 100000); % 定义一个频率范围，例如从 0 到 2

% 提取幅度和相位的数值函数
magnitude_1_num = abs(f_change_1_num(omega_values));
phase_1_num = angle(f_change_1_num(omega_values));
magnitude_2_num = abs(f_change_2_num(omega_values));
phase_2_num = angle(f_change_2_num(omega_values));

% 绘制幅度谱
figure;
subplot(2, 2, 1);
plot(omega_values, magnitude_1_num);
title('幅值 f_origin_1');
xlabel('Frequency (omega)');
ylabel('幅值');

% 绘制相位谱
subplot(2, 2, 2);
plot(omega_values, phase_1_num);
title('相位 f_origin_1');
xlabel('Frequency (omega)');
ylabel('相位');

% 绘制第二个信号的幅度谱
subplot(2, 2, 3);
plot(omega_values, magnitude_2_num);
title('幅值 f_origin_2');
xlabel('Frequency (omega)');
ylabel('幅值');

% 绘制第二个信号的相位谱
subplot(2, 2, 4);
plot(omega_values, phase_2_num);
title('相位 f_origin_2');
xlabel('Frequency (omega)');
ylabel('相位');
```

3、 用部分分式展开法求解反变换

1） 首先对分式进行表示，调用 residue 对部分分式进行展开，得到零点，极点和直接项

2） 根据部分分式分解结果来逐步进行傅里叶反变换，最终拼接在一起即可。

具体代码如下：

```
clear;clc;

%定义分式
```

```
B = conv([1 1], [1 4]);
A = conv([1 0], conv([1 2], [1 3]));

% 使用 residue 函数进行部分分式展开
[R, P, K] = residue(B, A);

% 打印部分分式展开结果
disp('零点:');
disp(R);
disp('极点:');
disp(P);
disp('直接项:');
disp(K);

% 反变换每个部分分式
syms t
h_t = 0; % 初始化 h(t)
for i = 1:length(R)
    h_t = h_t + R(i) * heaviside(t) * exp(P(i) * t);
end

% 如果有直流项，添加到 h(t)
if ~isempty(K)
    h_t = h_t + K(1) * heaviside(t);
end

% 显示反变换结果
disp('反变换结果:');
disp(h_t);
```

4、 画出系统函数的零点、极点分布

1） 调用 roots()函数求解分式的分子零点和分母极点

2） 根据求得的零点和极点画图。

3） 如果极点的实部均小于 0 则系统稳定，反之则不稳定。

具体代码如下:

```
clear;clc;

%定义分式
B = [1 2];
A = [1 1 2 6];

% 使用 roots 函数求解分子和分母的零点和极点
zeros = roots(B); % 分子的零点
poles = roots(A); % 分母的极点

% 创建传递函数
sys = tf(B, A);

% 绘制零极点图
figure;
pzmap(sys);
title('Pole-Zero Map of the System H(s)');
xlabel('Real Part');
ylabel('Imaginary Part');
grid on;
```

```
% 打印零点和极点
disp('Zeros of the system:');
disp(zeros);
disp('Poles of the system:');
disp(poles);

% 检查极点的位置以判断系统稳定性
if all(real(poles) < 0)
    disp('系统是稳定的，因为所有极点都位于左半平面。');
else
    disp('系统是不稳定的，因为存在极点不在左半平面。');
end
```

5、 求解冲击响应、阶跃响应和零状态响应

1）先把分式表示出来，对该分式直接进行拉普拉斯变换即可得到冲击响应

2）阶跃响应是冲击响应的积分，时域积分，频域乘以 1/s 即可得到频域的阶跃响应，对频域的阶跃响应进行拉普拉斯反变换即可得到阶跃函数。

3）调用 lsim 函数即可求得给定激励函数的输出，最后画图即可。

具体代码如下：

```
clear; clc;
syms s t

% 定义传递函数的分子和分母多项式
num = [1 0];        % s
den = [1 3 2];      % s^2 + 3s + 2

% 创建传递函数
sys = tf(num, den);

% 计算冲激响应的拉普拉斯反变换
H_s = poly2sym(num, s) / poly2sym(den, s);
h_t = ilaplace(H_s, s, t);
disp('Impulse response h(t):');
disp(h_t);

% 计算阶跃响应的拉普拉斯反变换
U_s = 1 / s;  % 拉普拉斯变换后的单位阶跃函数
Y_s = H_s * U_s;  % 传递函数乘以单位阶跃函数的拉普拉斯变换
y_t = ilaplace(Y_s, s, t);
disp('Step response y(t):');
disp(y_t);

% 冲激响应图
figure;
impulse(sys);
title('Impulse Response');
grid on;

% 阶跃响应图
figure;
step(sys);
title('Step Response');
grid on;
```

```
% 给定激励下的零状态响应
% 定义激励信号
t = 0:0.01:10;
u = cos(20*t).*heaviside(t);  % 例如一个正弦激励信号

% 计算零状态响应
[y, t_out] = lsim(sys, u, t);

% 绘制零状态响应图
figure;
plot(t_out, y);
title('Zero State Response to sin(t) Input');
xlabel('Time (s)');
ylabel('Response');
grid on;
```

## 二、实验结果

1、

```
时域信号：
exp(-2*abs(t))

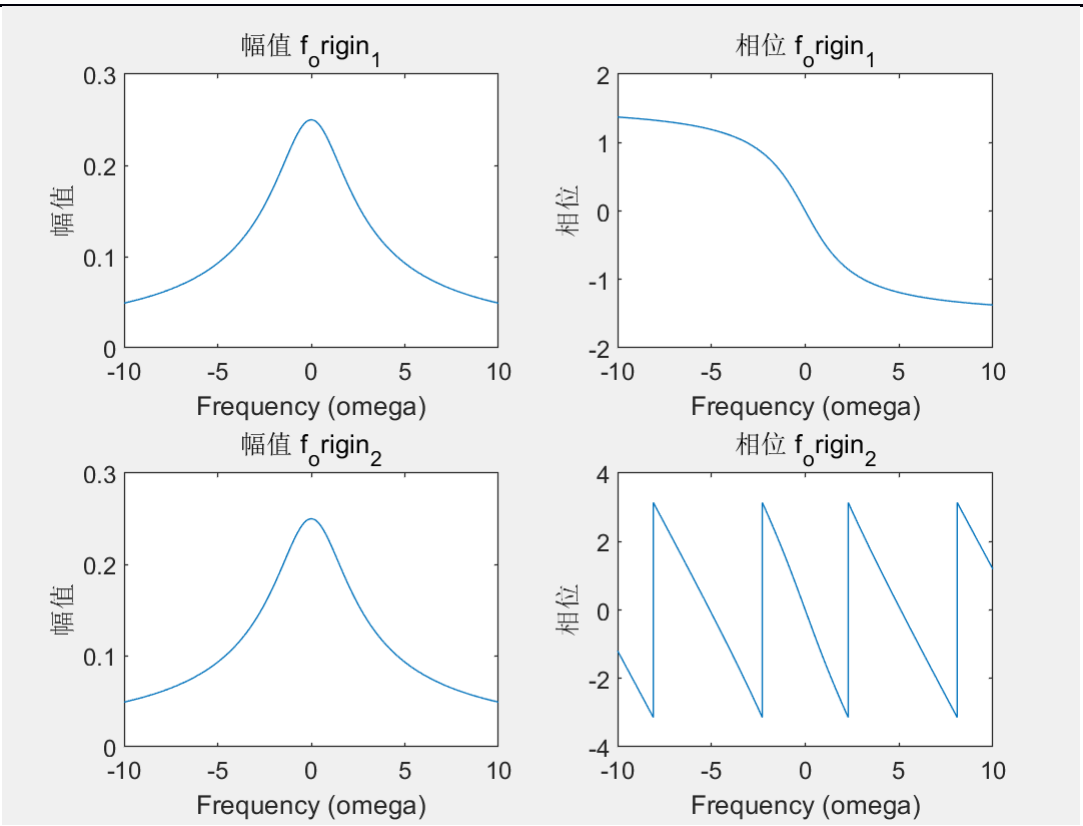
傅里叶变换后频域信号：
4/(omega^2 + 4)
```

命令行窗口

```
频域信号：
1/(omega^2 + 1)

傅里叶反变换后时域信号：
exp(-abs(t))/2
```

2、



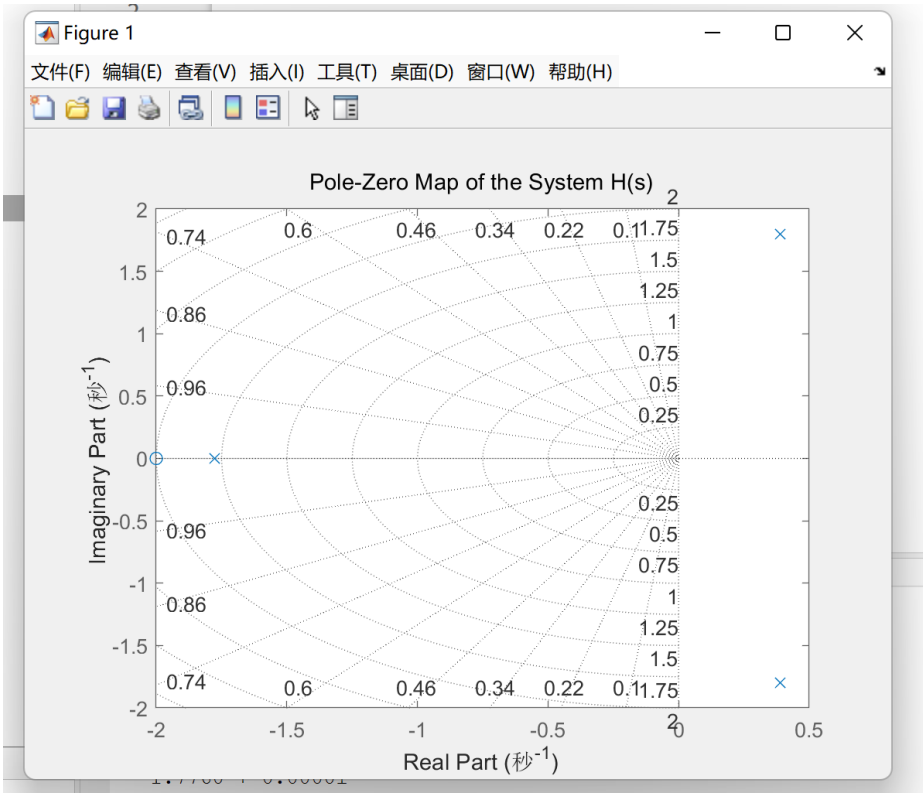

3、

直接项：

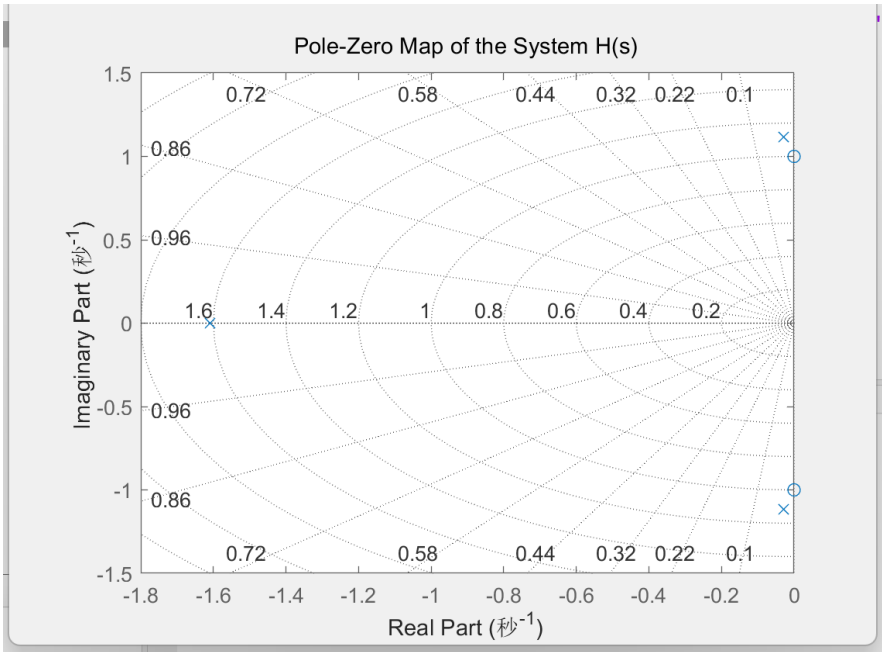
反变换结果：

```
(2*heaviside(t))/3 + exp(-2*t)*heaviside(t) - (2*exp(-3*t)*heaviside(t))/3
```

4、



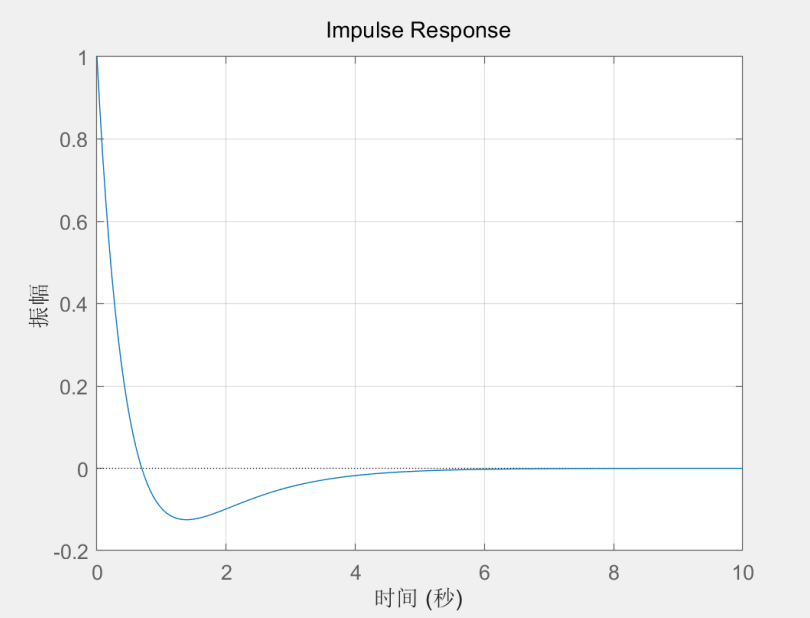

系统是不稳定的，因为存在极点不在左半平面。

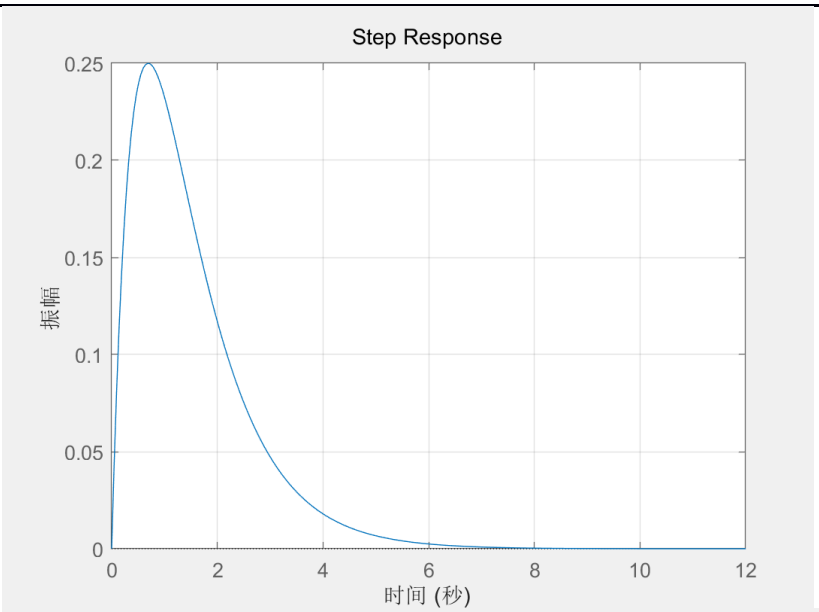


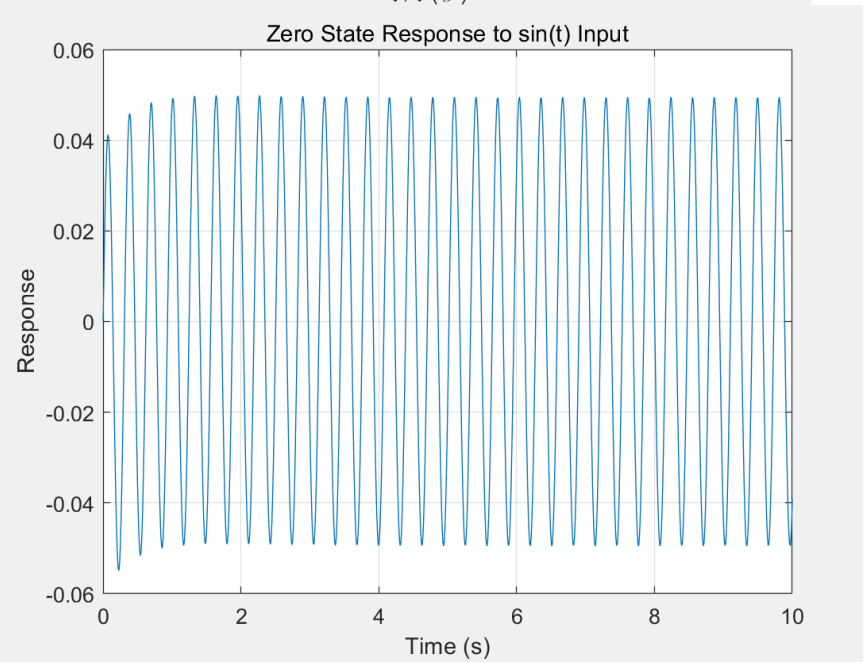

);

系统是稳定的，因为所有极点都位于左半平面。

5、






## 三、实验结果分析

1、这些问题都可以实现算出来，可以根据算的结果来判断代码是否出现问题。

2、问题 2 涉及到傅里叶变换的时移性质，从最后画的变换前后的频谱图来看，幅度没有变换，相位变化较大，因为只时移了单位 1，e^jw 模长为 1，对幅度没有影响，而对相位会有影响。

3、问题 3 部分分式展开，直接用拉普拉斯反变换应该也能算，用函数进行分式展开后需要从分式进行反变换，代码编写逻辑要清除。

4、问题 4 根据系统稳定性的判定条件来判断即可。

5、问题 5 需要对阶跃响应、冲击响应、时域积分性质理解才能够好地完成。

## 实验总结：

**心得：**

1、对 Matlab 求解傅里叶变换、傅里叶反变换、拉普拉斯变换、拉普拉斯反变换有了更深　刻的理解和应用。

2、 对傅里叶变换的时移性质、时域积分性质等有了更深的理解。

3、 加强了对这块一些傅里叶变换、拉普拉斯变换的一些计算转换的练习。

**碰到的问题和解决方法**:

1、 刚开始用的 python 但是直接调库的话出来的答案和实际有差别，w=2*pi*f 造成的，越搞越麻烦然后就转到 Matlab 上面，直接调用库函数就轻易出结果了。

2、 编程逻辑上还要加强。

## 参考文献

1、Matalb 里面 ifourier 函数用法

https://wenku.csdn.net/answer/bcd2d6137768646f03c0389d77837bb1

2、Matlab 里 fourier 函数用法

https://blog.csdn.net/weixin_44985601/article/details/104350131